

Scientific Description of Three ADDA-CUDA Extensions:

Multilayer Onion Geometry, Iterative Solvers, and Lanier Preconditioners

1 Iterative solvers in ADDA-CUDA

1.1 General CUDA solver architecture

ADDA solves the linear system

$$A p = b, \tag{1}$$

where A is the DDA interaction operator, p is the unknown dipole-polarization vector, and b is the incident-field vector. The iterative method is selected with

```
-iter <solver>
```

In ADDA-CUDA, the mathematical recurrences remain in the ADDA C solver source. During a CUDA solve, the large Krylov vectors remain GPU-resident, matrix-vector products use the CUDA `MatVec_GPU` path, scalar products and norms are evaluated with cuBLAS, and the elementary vector recurrences are executed by CUDA kernels. This avoids repeated host/device transfers of the large vectors during the iterative loop.

The current branch contains the historical ADDA solvers together with additional stabilized BiCG families. In particular, **BiCGStab(4)**, **BiCGStab(8)**, and **BiCGStab(12)** were added to ADDA in this development line to investigate and improve robustness on difficult, high-contrast, non-normal DDA systems.

1.2 Description of all available solvers

Command	Method	Description
<code>bcgs2</code> / <code>bicgstab2</code>	BCGS2 BiCGStab(2)	/ ADDA's enhanced two-step stabilized BiCG method. <code>bicgstab2</code> is an alias of <code>bcgs2</code> . Compared with classical BiCGStab, the degree-two stabilization polynomial can damp a broader part of the difficult spectrum while retaining moderate storage.

Command	Method	Description
<code>bicg</code>	BiCG_CS	Bi-Conjugate Gradient formulation specialized for complex-symmetric matrices. It exploits the complex-symmetric structure of the standard DDA operator and requires relatively little workspace, but it does not include the additional minimal-residual stabilization of the BiCGStab(L) family.
<code>bicgstab</code>	BiCGStab	Classical BiCGStab, corresponding to $L = 1$. It combines the BiCG recurrence with one stabilization step per cycle. Its memory footprint is small and it is often efficient, but difficult non-normal or high-contrast DDA systems may show oscillation or stagnation.
<code>bicgstab4 / bicgstabl4</code>	BiCGStab(4)	Added to ADDA in this branch. Four-step BiCGStab(L) stabilization. A degree-four minimal-residual polynomial is constructed in each outer cycle, giving stronger spectral damping than $L = 1$ or $L = 2$, at the price of additional Krylov vectors and matrix-vector products. This is the highest-order BiCGStab(L) variant retained for CUDA single-precision calculations; in that mode, <code>-recalc_resid</code> is recommended to verify the final physical residual.
<code>bicgstab8 / bicgstabl8</code>	BiCGStab(8)	Added to ADDA-CUDA in this branch. Eight-step stabilization with a deeper minimal-residual polynomial and a larger GPU-resident workspace. It was introduced primarily to enlarge the convergence domain. In the present implementation it is restricted to double precision. The higher-order correction polynomial is too sensitive to round-off for a reliable float32 implementation. Its computational cost is typically about twice that of BiCGStab(4). On the difficult multilayer benchmark used here it converges where BiCGStab(4) stagnates.
<code>bicgstab12 / bicgstabl12</code>	BiCGStab(12)	Added to ADDA-CUDA in this branch. Twelve-step stabilization. The larger polynomial and Krylov subspace can suppress spectral components that remain problematic for smaller L , although intermediate residual excursions can be strong. In the present implementation it is restricted to double precision. The order-12 correction and orthogonalization/minimal-residual steps are numerically too demanding for reliable float32 arithmetic. Its computational cost is typically about four times that of BiCGStab(4). On the reference high-contrast <code>onion</code> case, it is the most robust of the tested $L = 4, 8, 12$ variants.

Command	Method	Description
<code>cgnr</code>	CGNR	Conjugate Gradient applied to the normal equations associated with the residual minimization problem. It is broadly applicable because the normal-equation operator is Hermitian positive semidefinite, but forming the normal-equation iteration effectively squares the condition number and can therefore be unattractive for severely ill-conditioned DDA systems.
<code>csym</code>	CSYM	Conjugate-gradient-like method designed for complex-symmetric systems. It exploits the transpose symmetry of the standard DDA matrix rather than Hermitian symmetry. It is economical in storage but may lose its advantage when the operator departs from the complex-symmetric class.
<code>gpbicgstab2 / gpbicgstab12</code>	GPBiCGStab(2)	Generalized product-type BiCGStab method with two stabilization stages. It augments the BiCG recurrence with additional product-type recurrences intended to improve convergence on difficult non-symmetric or non-normal systems.
<code>gpbicgstab4 / gpbicgstab14</code>	GPBiCGStab(4)	Four-stage generalized product-type BiCGStab extension. It uses a larger recurrence and workspace than GPBiCGStab(2) and provides a stronger stabilization mechanism, at a correspondingly larger cost per outer iteration.
<code>qmr</code>	QMR_CS	Quasi-Minimal Residual method specialized for complex-symmetric matrices. QMR smooths the BiCG-type convergence through a quasi-minimization procedure and is the default iterative method in the base ADDA parameter set.
<code>qmr2</code>	QMR_CS_2	Second complex-symmetric QMR formulation supplied by ADDA. It uses a different recurrence and workspace organization from <code>qmr</code> , providing an alternative QMR path for the same general matrix symmetry class.

1.3 BiCGStab(4), BiCGStab(8), and BiCGStab(12)

The main solver extension developed here is the progression

$$\text{BiCGStab}(4) \longrightarrow \text{BiCGStab}(8) \longrightarrow \text{BiCGStab}(12). \quad (2)$$

Increasing L increases the degree of the stabilization polynomial used to minimize the residual over the local Krylov information. This increases the number of vectors, scalar products, and matrix-vector products per outer iteration, but the objective of these variants is not primarily a reduction of wall-clock time. The objective is to increase the probability of convergence for difficult DDA systems whose spectra cause lower-order methods to stagnate or undergo repeated unstable excursions.

All large $\text{BiCGStab}((L))$ vectors are kept GPU-resident during the iterative loop. Precision, however, becomes increasingly important as (L) grows. $\text{BiCGStab}(4)$ can still be used in the CUDA single-

precision executable, where the large vectors and FFT data are stored in `float32`. By contrast, BiCGStab(8) and BiCGStab(12) involve longer recurrences and higher-order stabilization polynomials, making them more sensitive to round-off errors. In the present implementation, their numerical stability is not considered sufficiently reliable in single precision. Therefore, **BiCGStab(8) and BiCGStab(12) are not recommended for single-precision calculations and should be used with the double-precision CUDA executables.**

This robustness has a computational cost. Relative to BiCGStab(4), BiCGStab(8) is typically about two times slower and BiCGStab(12) about three times slower for comparable problems. These higher-order solvers are therefore intended primarily for difficult systems for which convergence robustness is more important than raw execution speed.

1.4 Residual reliability, single precision, and the `-recalc_resid` option

Single-precision Krylov iterations are more sensitive to round-off accumulation and to a progressive difference between the recursively updated residual and the true residual of the physical DDA equations. In the present ADDA-CUDA implementation, the option

`-recalc_resid`

therefore has two complementary roles: an *in-iteration reliable-residual control* and a *final independent physical-residual validation*.

1.4.1 Periodic true-residual control and Krylov reset

When `-recalc_resid` is specified, ADDA-CUDA independently recomputes the physical residual

$$r_{\text{true}} = b - Ap \quad (3)$$

with the complete physical DDA operator every 20 iterations. The same check is also performed whenever the recursively propagated Krylov residual claims convergence, even if that iteration is not a multiple of 20.

The independently recomputed residual vector is compared with the residual carried by the Krylov recurrence. Defining the relative vector gap schematically as

$$g = \frac{\|r_{\text{true}} - r_{\text{rec}}\|}{\|r_{\text{true}}\|}, \quad (4)$$

the generic `-recalc_resid` policy uses a gap threshold of 10^{-2} . If the gap reaches or exceeds this value, or if the recursive residual indicates convergence while the physical residual does not, the Krylov recurrence is restarted from the independently recomputed true residual. The current solution vector is preserved; only the Krylov recurrence/history is rebuilt.

Thus `-recalc_resid` is no longer only an end-of-run diagnostic in ADDA-CUDA. It actively limits residual drift during the iterative solve. This is particularly important for BiCGStab(4) in single precision, for which the higher-order stabilization recurrence can accumulate significantly more round-off error than the lower-order methods.

The Lanier FULL, NESTED, and MULTIZONE paths retain their stronger reliable-residual policy. In addition to the periodic 20-iteration checks and the vector-gap test, these paths use the mandatory restart after iteration 3 and the ratio- $R = 100$ restart described in Part III.

1.4.2 Mandatory final residual check in single precision

At the end of the solve, ADDA recomputes the normalized physical residual

$$\frac{\|b - Ap\|}{\|b\|} \quad (5)$$

from a new matrix-vector product. This final value is independent of the recursively propagated Krylov residual. In the present implementation this final physical-residual check is **automatic for every single-precision calculation**, even when `-recalc_resid` is not explicitly supplied. In double precision, the same final check is performed when `-recalc_resid` is requested.

The distinction is therefore important:

- in single precision, the *final* physical residual is always checked;
- adding `-recalc_resid` also activates the *periodic* 20-iteration checks and the Krylov reset when residual drift becomes too large;
- in double precision, `-recalc_resid` activates both the periodic reliability control and the final independent residual check.

For this reason, the recommended form for a CUDA single-precision BiCGStab(4) calculation is

```
-iter bicgstab4 -recalc_resid
```

The periodic true-residual checks and restart mechanism provide the numerical control needed for practical use of the $L = 4$ recurrence in float32. BiCGStab(8) and BiCGStab(12), whose higher-order corrections are substantially more sensitive to finite-precision errors, remain restricted to the double-precision executable.

2 Part III — Preconditioning in ADDA-CUDA and Lanier preconditioners

The Lanier preconditioners implemented here are based on Steven Lanier, “Learning to precondition: Reinforcement learning enhanced three-level circulant preconditioning for the Discrete Dipole Approximation,” *Journal of Quantitative Spectroscopy and Radiative Transfer* 350 (2026), 109741.

The repository directory `ai/` contains the trained actor file `lanier_tqc_v1_ema_actor.bin`, created by Steven Lanier. The file provides the learned parameters used to configure the TQC-v1 Lanier preconditioner for the current scattering problem.

2.1 Preconditioners: general principle

For a difficult DDA system, the choice of iterative solver is only one part of the convergence problem. A preconditioner is an auxiliary operator that transforms the residual into a correction that is easier for the Krylov solver to exploit. In the notation used below, B denotes the operator actually applied to a residual,

$$z = B r, \quad (6)$$

with the ideal objective

$$B \approx A^{-1}, \quad (7)$$

where A is the exact DDA interaction operator. The purpose of preconditioning is not to modify the physical scattering problem, but to improve the spectral properties seen by the iterative method and thereby reduce stagnation, large residual excursions, and sensitivity to high refractive-index contrast.

A useful ADDA preconditioner should be substantially cheaper to apply than an exact inverse, preserve the complete physical DDA matrix in the Krylov matrix-vector products, remain stable for the target geometry under consideration, and fit within the available GPU memory. The unpreconditioned option

`-precon none`

therefore remains an important reference case.

2.2 Lanier preconditioners: common construction and practical choice

The Lanier-type preconditioners implemented in ADDA-CUDA approximate the discrete-dipole interaction operator by circulant or block-circulant models. Their principal computational advantage is that a circulant operator can be diagonalized in Fourier space and inverted efficiently using three-dimensional fast Fourier transforms (FFTs). The six independent components of the symmetric electromagnetic Green tensor,

$$G_{xx}, \quad G_{xy}, \quad G_{xz}, \quad G_{yy}, \quad G_{yz}, \quad G_{zz}, \quad (8)$$

are represented in the circulant construction. For each Fourier mode, the electromagnetic block reduces to a complex symmetric 3×3 matrix that can be inverted during initialization and reused during every application of the preconditioner.

Several Lanier variants were implemented during development. The simple `lanier/lanier1x` and `lanier15x/lanier1.5x` versions are useful as basic/reference circulant implementations, but the four practically useful configurations retained here are

Preconditioner	Geometry / physical situation	Typical example
<code>lanier_full</code>	One object with one refractive index	Homogeneous high-index sphere
<code>lanier_partition</code>	Two spatially separated objects	Two separated spheres (<code>bisphere2</code>)
<code>lanier_nested</code>	Two strongly coupled or nested regions with a large contact interface	Core-shell particle
<code>lanier_multizone</code>	Several geometric zones, but currently only two distinct refractive-index values	Multilayer <code>onion</code> with the two indices repeated among the layers

These four modes correspond to increasingly structured descriptions of the geometry. In all cases, the exact global DDA matrix-vector product remains unchanged; only the approximate inverse used by the Krylov solver is modified.

2.3 Full Lanier/TQC preconditioner: `lanier_full`

`lanier_full` is the principal Lanier preconditioner for a **single homogeneous object**, especially when the refractive index is high and the unpreconditioned system is difficult. A typical example is

a homogeneous sphere with a large real refractive index.

The method constructs one global circulant approximation of the complete object. In addition to the circulant representation of the Green tensor, it uses a transformed formulation involving the local dipole polarizability. Schematically,

$$\tilde{A} = I + SDS, \quad (9)$$

where D represents the electromagnetic interaction operator and S contains local factors derived from the dipole polarizability. The corresponding approximate inverse may be represented schematically as

$$B_{\text{full}} \simeq S^{-1}M^{-1}S^{-1}, \quad (10)$$

where M is the Fourier-diagonalizable circulant approximation.

The implementation incorporates the TQC-v1 parameterization. Parameters controlling the effective polarizability, Green-function wavenumber, local radial displacement, point/voxel-averaged Green-function blending, and related quantities are obtained from the trained Lanier actor model stored in

`lanier_tqc_v1_ema_actor.bin`

The actor therefore adapts the global circulant model to the physical characteristics of the scattering problem.

The essential limitation is that `lanier_full` uses *one global circulant inverse*. It is therefore best adapted to a single-index object. It can be applied formally to a heterogeneous particle, but the quality of one global approximation may degrade strongly when very different material regions are present.

Example: high-index homogeneous sphere. A representative command line is

```
adda_cuda.exe -grid 128 -shape sphere -m 3.0 0 \
  -iter bicgstab12 -eps 5 -recalc_resid -precon lanier_full
```

The numerical values are illustrative; the important point is the use of one connected object and one refractive-index value.

2.4 Lanier partition preconditioner: `lanier_partition` / `lanier_part`

`lanier_partition` is intended for **two spatially separated objects**. The canonical example is a pair of separated spheres. Each object receives its own regional Lanier inverse, while the electromagnetic interaction between the two objects remains present in the exact global ADDA matrix-vector product.

For two objects, the complete system may be written

$$\begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \begin{pmatrix} p_1 \\ p_2 \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \end{pmatrix}. \quad (11)$$

The regional preconditioners approximate the diagonal blocks A_{11} and A_{22} . The off-diagonal couplings A_{12} and A_{21} are not removed from the physical problem.

The method is most appropriate when the two objects are genuinely separated, because each object then has strong internal interactions while the inter-object coupling is comparatively weaker. It is not the preferred formulation for a core-shell particle or for two regions sharing a large interface.

For this purpose ADDA-CUDA provides the two-material geometry **bisphere2**. The parameter is the center-to-center distance divided by the sphere diameter,

```
-shape bisphere2 <R_cc/d>
```

and the two spheres are represented by two distinct ADDA material domains. Thus, two refractive-index values supplied by **-m** are both used by the geometry, as required by **lanier_partition**. For $R_{cc}/d > 1$, the two spheres are spatially separated.

Example: two separated spheres. For $R_{cc}/d = 1.5$, with refractive indices $3.0 + 0i$ and $1.2 + 0i$, the tested command is

```
E:\msna6\adda-cuda-main\cmake-build-debug\bin>adda_cuda.exe -grid 128 -shape bisphere2
↪ 1.5 -m 3.0 0 1.2 0 -iter bicgstab12 -eps 5 -recalc_resid -precon lanier_partition
```

This is the natural geometry class for the partition strategy: two separated connected objects represented by two distinct material domains, with no large contact interface.

2.5 Nested Lanier preconditioner: **lanier_nested** / **lanier_core_shell**

lanier_nested is designed for **two strongly coupled regions with a large contact interface**, or for one region geometrically contained inside the other. The canonical example is a core-shell particle. It should therefore be distinguished from **lanier_partition**, which is intended for separated objects.

Let M_1 and M_2 denote binary masks identifying the two material regions. Two independent FULL6/TQC circulant inverses, B_1 and B_2 , are constructed. The current nested formulation uses an additive masked block-Jacobi operator,

$$B_{\text{nested}} = M_1 B_1 M_1 + M_2 B_2 M_2, \quad (12)$$

so that

$$z = M_1 B_1 (M_1 r) + M_2 B_2 (M_2 r). \quad (13)$$

The input and output masks are both required because the FFT-based regional convolution is nonlocal. They prevent a correction generated by one regional inverse from leaking into the other material region.

Although the approximate inverse is block-separated, the exact Krylov matrix-vector product always uses the complete operator

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}, \quad (14)$$

so the physical electromagnetic coupling across the core-shell interface is fully retained.

Example: two-domain core-shell particle. The general **onion** geometry provides a convenient two-domain core-shell target. For example,

```
adda_cuda.exe -iter bcgs2 -eps 8 -recalc_resid -precon none -grid 64 \
-shape coated 0.60 -m 1.5 0 2.0 0
```

or for a more difficult example with " **bicgstab12** "

```
adda_cuda -grid 128 -shape onion 0.50 -m 1.2 0 3.0 0 -iter bicgstab12 \
-eps 5 -recalc_resid -precon lanier_nested
```

defines an outer shell (domain 1) and a core of diameter $0.5d$ (domain 2). The two regions share a complete spherical interface, which is precisely the strongly coupled situation targeted by `lanier_nested`.

2.6 Multizone Lanier preconditioner: `lanier_multizone` / `lanier_mz`

`lanier_multizone` extends the masked regional approach to **an arbitrary number of geometric zones**, but the **current implementation is restricted to only two distinct refractive-index values**. These two optical indices may be repeated over many ADDA domains. A multilayer onion with alternating indices is the reference example.

Let

$$\Omega = \bigcup_{j=1}^{N_z} \Omega_j \quad (15)$$

be a decomposition into N_z disjoint ADDA domains and let M_j denote the mask of zone j . An independent FULL6/TQC Lanier inverse B_j is constructed for each zone. The multizone operator is

$$B_{MZ} = \sum_{j=1}^{N_z} M_j B_j M_j, \quad (16)$$

and therefore

$$z = B_{MZ} r = \sum_{j=1}^{N_z} M_j B_j (M_j r). \quad (17)$$

A zone is identified by its ADDA domain number, not only by its refractive-index value. Consequently,

$$m_A, m_B, m_A, m_B, \dots \quad (18)$$

may represent many independently preconditioned geometric regions even though the complete particle contains only the two optical indices m_A and m_B .

2.6.1 V2 implementation: strict masks and shared FFT workspace

For every occupied zone, the CUDA V2 implementation determines a tight Cartesian bounding box, constructs an auxiliary grid typically close to a $1.5\times$ embedding, builds the six independent Fourier-space tensor components (FULL6), evaluates the TQC/actor parameters, and retains the corresponding per-mode 3×3 inverse coefficients on the GPU.

The large temporary FFT grid and cuFFT workspace are not duplicated for all zones. One workspace, sized for the largest regional problem, is shared sequentially. This substantially reduces GPU memory usage when the number of zones is large.

The current actor output includes parameters such as

$$\alpha_{\text{opt}}, \quad K_S, \quad L_S, \quad w_{\text{blend}}, \quad r_{\text{shift}}, \quad (19)$$

which tune the effective local circulant model. They are computed independently for each ADDA domain. Hence two layers carrying the same refractive index may still receive different regional conditioners because their geometries and bounding boxes differ.

2.6.2 Relation to the exact DDA operator

The regional decomposition affects only the approximate inverse. The exact global matrix-vector product always contains all intra-zone and inter-zone electromagnetic interactions,

$$A = \begin{pmatrix} A_{11} & A_{12} & \cdots & A_{1N} \\ A_{21} & A_{22} & \cdots & A_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ A_{N1} & A_{N2} & \cdots & A_{NN} \end{pmatrix}. \quad (20)$$

The preconditioner approximates only the diagonal regional inverses,

$$B_{MZ} \approx \begin{pmatrix} A_{11}^{-1} & 0 & \cdots & 0 \\ 0 & A_{22}^{-1} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & A_{NN}^{-1} \end{pmatrix}. \quad (21)$$

There is no global residual update between zones during one application of the regular multizone preconditioner; the zones are combined additively. Cross-zone coupling re-enters immediately through the next exact global ADDA matrix-vector product.

Example: four-layer onion with two alternating indices. A representative multizone command is

```
adda_cuda -grid 128 -shape onion 0.72 0.48 0.24 -m 1.2 0 2.5 0 1.2 0 2.5 0 \
-iter bicgstab4 -eps 8 -maxiter 3000 -recalc_resid -precon lanier_multizone
```

or for a more difficult object with "bicgstab12"

```
adda_cuda -grid 128 -shape onion 0.72 0.48 0.24 -m 1.2 0 3.0 0 1.2 0 3.0 0 \
-iter bicgstab12 -eps 5 -recalc_resid -precon lanier_multizone
```

This target has four geometric zones but only two distinct refractive indices, 1.2 and 3.0, repeated alternately. It illustrates both the purpose and the present limitation of `lanier_multizone`. The implementation can handle many ADDA zones (up to the compiled domain limit), but only two distinct optical-index values are currently supported by this preconditioner.

2.7 Summary of the four useful Lanier configurations

The practical selection rule is therefore

Mode	Recommended use
<code>lanier_full</code>	One connected object with one refractive index; especially useful for a difficult high-index homogeneous target such as a sphere.
<code>lanier_partition</code>	Two separated objects; typical example: two separated spheres.
<code>lanier_nested</code>	Two strongly coupled, touching, or nested material regions with a large interface; typical example: core-shell sphere.
<code>lanier_multizone</code>	Several geometric zones using only two distinct refractive indices, which may repeat between zones; typical example: multilayer onion.

The simpler `lanier/lanier1x` and `lanier15x/lanier1.5x` implementations remain useful as development and reference circulant models, but they are not the principal practical configurations emphasized in this document.

2.8 Reliable residual and restart policy for Lanier-preconditioned solvers

The reliable-residual mechanism is a solver-side safeguard and applies to all Lanier preconditioners; it is not specific to `lanier_multizone`. In particular, it is used with the four practical configurations described above: `lanier_full`, `lanier_partition`, `lanier_nested`, and `lanier_multizone`. The Lanier FULL/NESTED/MULTIZONE paths use a stronger automatic policy than the generic `-recalc_resid` mechanism: the same periodic true-residual verification is supplemented by mandatory and ratio-based resets. The simpler Lanier reference modes obtain the generic periodic control when `-recalc_resid` is specified.

For the difficult CUDA single-precision calculations considered here, the large solver vectors and FFT data remain in `float32`, whereas scalar products are accumulated in FP64 and the recurrence coefficients are retained as double-precision complex scalars. Reliable checks periodically recompute the physical residual

$$r_{\text{true}} = b - Ap \tag{22}$$

with the exact global ADDA matrix-vector product. Consequently, convergence is never accepted solely from the recursively propagated Krylov residual.

The validation policy used for the robust `BiCGStab(L)` calculations consists of

- one mandatory true-residual check and restart after iteration 3;
- a true physical residual evaluation every 20 iterations;
- an additional ratio- R reliable restart with $R = 100$;
- a restart if the vector-gap criterion reaches 10^{-2} ;
- acceptance of convergence only when the recursive convergence criterion is confirmed by the true physical residual.

This policy is particularly useful with higher-order `BiCGStab(L)` methods because large temporary residual excursions can occur without implying final divergence. The reliable checks distinguish genuine changes of the physical residual from drift of the recursively propagated residual and provide controlled restart points when required.